专升本 **Java** 程序设计模拟卷-卷 **10**

fifine-专升本

2026 年 3 月 2 日

一、单项选择题（本大题共 **20** 小题，每小题 **2** 分，共 **40** 分）

1. 下列哪项是 **Java** 语言的特性？

A. 指针运算

B. 多重继承

C. 自动内存管理

D. 预处理器

答案与深度解析 答案：**C.** 自动内存管理

【核心认知框架】

- 题目本质：考查 Java 语言核心设计哲学与 C/C++ 的区别
- 关键区分点：内存安全 vs 性能控制，单继承 vs 多实现
- 命题人意图：测试对 Java 安全特性的理解，排除 C++ 遗留概念

【选项原理深度拆解】

2. **A.** 指针运算—错误（安全机制）
   - **JLS** 规范：JLS §4.12.4 明确移除指针算术运算
   - 安全理由：防止缓冲区溢出、非法内存访问
   - 字节码证据：JVM 指令集无 `ptr_add` 等指令，引用通过句柄或直接指针管理但对用户透明
   - 例外：JNI 中可通过 C 代码操作指针，但纯 Java 不行
3. **B.** 多重继承—错误（菱形问题）
   - 设计决策：Java 类支持单继承，接口支持多实现
   - 原因：避免 C++ 菱形继承的二义性问题
   - 字节码证据：类文件结构中 `super_name` 只有一个父类索引
   - 替代：默认方法（Java 8）提供类似多重继承行为
4. **C.** 自动内存管理—正确（**GC** 机制）
   - 核心机制：垃圾回收器（GC）自动回收无用对象
   - **JVM** 规范：JVMS §2.5 定义堆内存管理策略
   - 字节码证据：无 `free` 指令，对象生命周期由 GC 根引用链决定
   - 优势：防止内存泄漏（大部分情况），简化开发
5. **D.** 预处理器—错误（编译模型）
   - 编译流程：Java 无宏预处理器（如 C 的 #define）

- 替代：使用常量（`static final`）、泛型、注解
- 原因：保持代码可读性，避免宏带来的调试困难
- 例外：某些框架（如 Lombok）通过注解处理器模拟预处理

【应背诵知识点】

- Java 特性：简单、面向对象、分布式、健壮、安全、架构中立、可移植、解释型、高性能、多线程、动态
- 无指针运算、无多重继承（类）、无预处理器

【命题趋势与实战策略】

- 考场秒杀：看到指针、多重继承、预处理器直接排除
- 高频陷阱：混淆接口多实现与类多重继承

6. 在 **Java** 中，哪个关键字用于引入包中的类？

A. package

B. import

C. include

D. using

答案与深度解析 答案：**B. import**

【核心认知框架】

- 题目本质：考查 Java 包管理机制与编译单元结构
- 关键区分点：定义包 vs 导入包 vs 其他语言语法
- 命题人意图：测试基础语法敏感度，区分 C/C++/C# 语法

【选项原理深度拆解】

7. **A. package** —错误（定义而非导入）
   - 功能：声明当前文件所属包，必须在文件首行（注释后）
   - 字节码：影响 class 文件常量池中的 `ThisClass` 全限定名
   - 错误用法：`package java.util;` 不能导入类，只能定义归属
8. **B. import** —正确（编译期指令）
   - 功能：告诉编译器类路径，简化类名书写
   - 字节码证据：import 不影响字节码，常量池仍存全限定名（如 `java/util/List`）
   - 机制：编译期语法糖，运行时 JVM 不知 import 存在
   - 静态导入：`import static` 可导入静态成员

9. **C. include** —错误（**C/C++** 语法）
   - 来源：C/C++ 预处理器指令，用于包含头文件
   - **Java** 对应：无直接对应，类加载机制替代头文件包含
   - 编译错误：Java 编译器不识别 `include`
10. **D. using** —错误（**C#/Python** 语法）
   - 来源：C# 命名空间导入，Python 模块导入
   - **Java** 对应：无此关键字
   - 混淆点：Java 7 try-with-resources 使用 `try (Resource r = ...)` 但非 using 关键字

   【应背诵知识点】
   - import 仅影响编译，不影响运行时
   - java.lang 包自动导入，无需显式 import
   - 同名类冲突需使用全限定名

   【命题趋势与实战策略】
   - 考场秒杀：Java 导入必选 import
   - 高频陷阱：混淆 package 和 import 顺序

11. 下列关于变量的默认值，错误的是？

答案与深度解析 答案：**D.** 局部变量有默认值 【核心认知框架】：考查变量初始化规则。【选项深度拆解】：
- **A/B/C**：正确。成员变量有默认值（int=0, boolean=false, ref=null）。
- **D**：错误。局部变量必须显式初始化，否则编译错误。
- 字节码：局部变量表未赋值前访问会报 `VerifyError` 或编译期报错。
- **JLS**：§4.12.5 规定局部变量必须初始化。

12. 下列运算符中，优先级最高的是？

答案与深度解析 答案：**C. ++** 【核心认知框架】：考查运算符优先级。【选项深度拆解】：
- **++**：最高（一元运算符）。
- *****：次之（乘法）。
- **+**：再次（加法）。
- **=**：最低（赋值）。
- 字节码：`iinc` 指令实现 ++，优先级影响表达式树构建。

13. 关于 **break** 和 **continue** 的说法，正确的是？

答案与深度解析 答案：**C. break 和 continue 都可以带标签** 【核心认知框架】：考查跳转控制。【选项深度拆解】：

- **A**：错误。break 可用于循环。
- **B**：正确。continue 仅用于循环。
- **C**：正确。标签用于多层循环跳转。
- **D**：错误。continue 不可用于 switch。
- 字节码：`goto` 指令实现跳转，标签对应字节码偏移量。

14. 下列哪个选项是正确的二维数组声明？

答案与深度解析 答案：**B. int[][] a = new int[3][];** 【核心认知框架】：考查数组语法。【选项深度拆解】：

- **A**：错误。Java 声明不能在左边指定大小。
- **B**：正确。第一维必须指定，第二维可后续分配。
- **C**：错误。new 时第一维必须指定。
- **D**：错误。new 时必须指定至少第一维。
- 内存：数组的数组，每个元素是引用。

15. 关于构造方法的描述，正确的是？

答案与深度解析 答案：**C. 构造方法可以重载** 【核心认知框架】：考查构造器特性。【选项深度拆解】：

- **A**：错误。无返回类型，连 void 都不能写。
- **B**：错误。构造器不可继承。
- **C**：正确。参数列表不同即可重载。
- **D**：错误。构造器不可重写（可隐藏）。
- 字节码：构造器名为 `<init>`，重载通过描述符区分。

16. 在 **Java** 中，使用哪个关键字表示当前对象的引用？

答案与深度解析 答案：**B. this** 【核心认知框架】：考查 this 语义。【选项深度拆解】：

- **A**：super 指父类。
- **B**：this 指当前对象。
- **C/D**：Java 无 self/current 关键字。
- 字节码：`aload_0` 指令加载 this 引用。

17. 下列哪个访问修饰符的成员在同一个包中可见，且对子类可见（无论子类在哪个包中）？

答案与深度解析 答案：**B. protected** 【核心认知框架】：考查访问控制权限。【选项深度拆解】：

- **A**：public 全局可见。
- **B**：protected 包内 + 子类可见。
- **C**：private 仅本类。
- **D**：默认仅包内。
- 字节码：ACC_PROTECTED 标志位控制访问检查。

18. 关于静态方法，以下说法正确的是？

答案与深度解析 答案：**D.** 静态方法可以通过类名直接调用 【核心认知框架】：考查 static 语义。【选项深度拆解】：

- **A**：错误。不可访问非静态成员。
- **B**：错误。无 this 上下文。
- **C**：错误。静态方法不可重写（可隐藏）。
- **D**：正确。`Class.method()`。
- 字节码：`invokestatic` 指令调用，无需对象引用。

19. 下列哪个类是所有 **Java** 异常类的父类？

答案与深度解析 答案：**D. Throwable** 【核心认知框架】：考查异常体系。【选项深度拆解】：

- **A/B/C**：均为 Throwable 子类。
- **D**：正确。根节点。
- 字节码：`athrow` 指令要求操作数为 Throwable 子类。

20. 关于 **try-catch-finally** 结构，以下说法正确的是？

答案与深度解析 答案：**B. finally** 块可以没有【核心认知框架】：考查异常结构。【选项深度拆解】：

- **A**：错误。父类异常必须放在子类之后。
- **B**：正确。try-catch 或 try-finally 均合法。
- **C**：错误。可只有 catch 或只有 finally。
- **D**：错误。catch 不能独立存在。
- 字节码：异常表定义捕获范围，finally 通过 jsr 或新机制实现。

21. 下列哪个集合类是有序的，且允许重复元素？

答案与深度解析 答案：**C. ArrayList** 【核心认知框架】：考查集合特性。【选项深度拆解】：

- **A/B**：Set 不可重复。
- **C**：正确。List 有序可重复。
- **D**：Map 是键值对。
- 底层：ArrayList 基于数组，索引有序。

22. 关于泛型，以下说法错误的是？

答案与深度解析 答案：**D.** 泛型可以用于基本数据类型 【核心认知框架】：考查泛型限制。【选项深度拆解】：

- **A/B/C**：正确。
- **D**：错误。泛型参数必须是引用类型（自动装箱）。
- 字节码：类型擦除后变为 Object，基本类型无法继承 Object。

23. 在 **Java** 多线程中，**start()** 方法和 **run()** 方法的区别是？

答案与深度解析 答案：**A. start()** 启动新线程并执行 **run()** 方法，**run()** 只是普通方法调用 【核心认知框架】：考查线程启动。【选项深度拆解】：

- **A**：正确。start 请求 JVM 创建线程。
- **B/C/D**：错误。
- 字节码：start 调用 native 方法创建线程，run 是普通 invokevirtual。

24. 关于线程同步，以下说法错误的是？

答案与深度解析 答案：**D.** 使用 **synchronized** 可以提高程序性能【核心认知框架】：考查同步开销。【选项深度拆解】：

- **A/B/C**：正确。
- **D**：错误。同步有开销，可能降低性能。
- 原理：获取锁、释放锁、上下文切换均有成本。

25. 在 **Java I/O** 中，**BufferedWriter** 类的作用是？

答案与深度解析 答案：**A.** 提供缓冲功能，提高字符写入效率 【核心认知框架】：考查 IO 缓冲。【选项深度拆解】：

- **A**：正确。减少 IO 调用次数。
- **B/C/D**：错误。
- 原理：内存缓冲区积累数据后一次性写入。

26. 关于序列化，**transient** 关键字的作用是？

答案与深度解析 答案：**A.** 标记变量为不可序列化 【核心认知框架】：考查序列化控制。【选项深度拆解】：
- **A**：正确。忽略该字段。
- **B/C/D**：错误。
- 字节码：序列化机制检查 ACC_TRANSIENT 标志。

27. 在反射中，获取 **Class** 对象的三种方式不包括？

答案与深度解析 答案：**D. new Class()** 【核心认知框架】：考查反射 API。【选项深度拆解】：
- **A/B/C**：正确。
- **D**：错误。Class 构造器私有。
- 原理：Class 对象由 JVM 创建，用户不可 new。

28. **JDBC** 中，**PreparedStatement** 与 **Statement** 相比，优势不包括？

答案与深度解析 答案：**D.** 可以执行任意复杂的 **SQL** 语句 【核心认知框架】：考查 JDBC 安全性。【选项深度拆解】：
- **A/B/C**：正确。
- **D**：错误。两者都能执行复杂 SQL，非 PreparedStatement 独有优势。
- 核心：预编译、防注入、批量。

29. **Java** 语言是由哪家公司开发的？

答案与深度解析 答案：**C. Sun Microsystems**【核心认知框架】：考查历史常识。【选项深度拆解】：
- **A/B/D**：错误。
- **C**：正确。1995 年 Sun 发布，2010 年 Oracle 收购。
- 证据：官方文档版权信息。

二、判断题（本大题共 10 小题，每小题 2 分，共 20 分）

1. **Java** 虚拟机（**JVM**）负责将字节码转换为特定平台的机器码。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查 JVM 核心功能。【深度解析】：
- 机制：解释器逐行解释，JIT 编译器编译为机器码。
- 跨平台：字节码统一，JVM 平台相关。

- 字节码：JVM 指令集独立于硬件指令集。

2. 在同一个类中，可以定义两个参数列表相同但返回值类型不同的方法。（　）

答案与深度解析 答案：× 【核心认知框架】：考查方法重载规则。【深度解析】：
- 规则：重载仅看参数列表，返回值不足以区分。
- 编译错误：`method is already defined`。
- 字节码：方法描述符包含返回值，但 JVM 加载时禁止签名冲突。

3. 子类可以访问父类的 **protected** 成员。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查访问控制。【深度解析】：
- 规则：protected 对子类和同包可见。
- 例外：不同包子类只能通过继承访问，不能通过父类实例访问。
- 字节码：访问检查通过 ACC_PROTECTED 标志。

4. 抽象类不能实例化，但可以有构造方法。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查抽象类特性。【深度解析】：
- 实例化：不能 new 抽象类。
- 构造器：供子类调用（super）。
- 字节码：抽象类有 `<init>` 方法，但类标志 ACC_ABSTRACT。

5. 接口中的默认方法（**default method**）可以有方法体。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查 Java 8 接口新特性。【深度解析】：
- **Java 8**：引入 default 方法，提供默认实现。
- 目的：接口演进不破坏实现类。
- 字节码：接口方法可有字节码指令，标志 ACC_PUBLIC | ACC_DEFAULT。

6. 使用 **throws** 关键字可以在方法内部抛出异常。（　）

答案与深度解析 答案：× 【核心认知框架】：考查异常关键字区别。【深度解析】：
- **throws**：方法签名声明，不抛出。
- **throw**：方法体内抛出异常对象。
- 字节码：`athrow` 对应 throw，异常表对应 throws。

7. **HashSet** 中的元素按插入顺序存储。（　）

答案与深度解析 答案：× 【核心认知框架】：考查集合有序性。【深度解析】：
- **HashSet**：无序（基于 HashMap）。
- **LinkedHashSet**：按插入顺序。
- **TreeSet**：按自然顺序/比较器。
- 原理：哈希表存储位置由 hashCode 决定。

8. 泛型类型参数在运行时会被擦除，因此无法在运行时获取泛型的具体类型。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查类型擦除。【深度解析】
- 机制：编译期擦除为 Object 或边界。
- 例外：反射可通过 GenericSuperclass 获取部分信息。
- 字节码：Signature 属性保留泛型信息，但运行时对象无泛型类型。

9. 线程调用 **wait()** 方法后，会释放持有的对象锁。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查线程通信。【深度解析】
- 行为：wait 进入等待集，释放 monitor。
- 对比：sleep 不释放锁。
- 原理：允许其他线程进入同步块通知。

10. **File** 类的 **exists()** 方法可以判断文件或目录是否存在。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查 File API。【深度解析】
- 功能：检查路径是否存在。
- 返回：boolean。
- 原理：调用操作系统 API 检查文件系统。

三、填空题（本大题共 5 小题，每空 2 分，共 20 分）

1. **Java** 的基本数据类型中，字符类型是 _____，布尔类型是 _____。

答案与深度解析 答案：**char**、**boolean** 【核心认知框架】：考查基本类型分类。【深度解析】
- **char**：16 位 Unicode 字符。
- **boolean**：true/false，底层多用 int 表示。
- 字节码：`char` 用 `iload` (int)，`boolean` 用 `iconst`。

2. 在面向对象中，_____ 是指一个类具有多种形态，包括方法重载和 _____。

答案与深度解析 答案：多态、方法重写 【核心认知框架】：考查 OOP 特征。【深度解析】
- 多态：编译时（重载）+ 运行时（重写）。
- 重写：子类覆盖父类方法，动态绑定。
- 字节码：`invokevirtual` 实现运行时多态。

3. 使用 _____ 关键字可以调用父类的构造方法，使用 _____ 关键字可以调用本类的其他构造方法。

答案与深度解析 答案：**super**、**this** 【核心认知框架】：考查构造器调用。【深度解析】
- **super**：`super()` 必须首行。
- **this**：`this()` 必须首行，与 super 互斥。
- 字节码：`invokespecial` 调用构造器。

4. 线程的五种状态包括新建、_____、运行、_____ 和死亡。

答案与深度解析 答案：就绪、阻塞（或等待/超时等待）【核心认知框架】：考查线程生命周期。【深度解析】
- 状态：NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED。
- 转换：start-> 就绪，调度-> 运行，wait/sleep-> 阻塞。
- **JVM**：线程映射到 OS 内核线程。

5. 集合框架中，**List** 接口的常用实现类有 _____ 和 _____。

答案与深度解析 答案：**ArrayList**、**LinkedList** 【核心认知框架】：考查集合体系。【深度解析】
- **ArrayList**：数组，随机访问快。
- **LinkedList**：链表，插入删除快。
- 选择：读多 ArrayList，写多 LinkedList。

四、程序运行结果题（本大题共 5 小题，每小题 6 分，共 30 分）

1. 写出下列程序的输出结果：

```java
public class Test1 {
public static void main(String[] args) {
```

```
int a = 5, b = 7;
while (a < b) {
a++;
b--;
}
System.out.println("a=" + a + ", b=" + b);
}
}
```

答案与深度解析 答案：**a=6, b=6** 【核心认知框架】：考查循环与变量变化。【执行流程分析】

- 初始：a=5, b=7。
- 循环 **1**：5<7 真，a=6, b=6。
- 循环 **2**：6<6 假，退出。
- 输出：a=6, b=6。
- 字节码：`if_icmpge` 实现条件跳转。

2. 写出下列程序的输出结果：

```
class Parent {
public void method() { System.out.print("Parent "); }
}
class Child extends Parent {
public void method() { System.out.print("Child "); }
}
public class Test2 {
public static void main(String[] args) {
Parent p = new Child();
p.method();
}
}
```

答案与深度解析 答案：**Child** 【核心认知框架】：考查多态动态绑定。【执行流程分析】

- 对象：p 编译类型 Parent，运行类型 Child。
- 调用：`p.method()` 动态绑定到 Child.method。
- 输出：Child。
- 字节码：`invokevirtual` 根据实际对象类型分派。

3. 写出下列程序的输出结果：

```java
public class Test3 {
public static void main(String[] args) {
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
System.out.println(s1 == s2);
System.out.println(s1 == s3);
}
}
```

答案与深度解析 答案：**true false**【核心认知框架】：考查字符串常量池。【执行流程分析】

- **s1/s2**：字面量，指向常量池同一对象，== 为 true。
- **s3**：new 创建堆新对象，== 为 false。
- 字节码：`ldc` 加载常量池，`new` 创建堆对象。

4. 写出下列程序的输出结果：

```java
public class Test4 {
public static void main(String[] args) {
int[] arr = {1, 2, 3, 4, 5};
int sum = 0;
for (int i = 0; i < arr.length; i += 2) {
sum += arr[i];
}
System.out.println(sum);
}
}
```

答案与深度解析 答案：**9**【核心认知框架】：考查数组遍历与步长。【执行流程分析】

- **i=0**：sum+=1 (1)。
- **i=2**：sum+=3 (4)。
- **i=4**：sum+=5 (9)。
- **i=6**：退出。
- 结果：9。
- 字节码：`iaload` 加载数组元素。

5. 写出下列程序的输出结果：

```java
class MyClass {
int x;
static int y;
MyClass(int x) {
this.x = x;
y++;
}
}
public class Test5 {
public static void main(String[] args) {
MyClass obj1 = new MyClass(5);
MyClass obj2 = new MyClass(10);
System.out.println(obj1.x + " " + obj2.x);
System.out.println(MyClass.y);
}
}
```

答案与深度解析 答案：

```
5 10
2
```

【核心认知框架】：考查静态变量共享性。【执行流程分析】

- **x**：实例变量，各自独立（5, 10）。
- **y**：静态变量，共享累加（1+1=2）。
- 输出：5 10 换行 2。
- 字节码：`getfield` 访问 x，`getstatic` 访问 y。

五、编程题（本大题共 2 小题，每小题 20 分，共 40 分）

1. 设计一个" 几何图形" 类及其子类

答案与深度解析 答案：见代码实现 【核心认知框架】：考查抽象类、多态、数组遍历。【设计思路深度解析】

- 抽象类 **Shape**：定义 area/perimeter 抽象方法，规范接口。
- 子类 **Circle/Rectangle**：实现具体计算逻辑，封装属性。
- 多态：`Shape[]` 数组存储子类，统一调用方法。
- 内存模型：数组存引用，对象存堆，动态绑定。

- 最佳实践：遵循开闭原则，易于扩展新图形。

【参考代码】

```java
abstract class Shape {
    public abstract double area();
    public abstract double perimeter();
}

class Circle extends Shape {
    private double radius;
    public Circle(double r) { radius = r; }
    public double area() { return Math.PI * radius * radius; }
    public double perimeter() { return 2 * Math.PI * radius; }
}

class Rectangle extends Shape {
    private double l, w;
    public Rectangle(double l, double w) { this.l = l; this.w = w; }
    public double area() { return l * w; }
    public double perimeter() { return 2 * (l + w); }
}

public class Main {
    public static void main(String[] args) {
        Shape[] shapes = new Shape[4];
        shapes[0] = new Circle(1); shapes[1] = new Circle(2);
        shapes[2] = new Rectangle(3, 4); shapes[3] = new Rectangle(5, 6);
        for (Shape s : shapes) {
            System.out.println("Area: " + s.area() + ", Perimeter: " + s.perimeter());
        }
    }
}
```

【考点延伸】

- 多态：`s.area()` 运行时绑定具体实现。
- 抽象：强制子类实现计算逻辑。
- 扩展：新增 Triangle 类只需继承 Shape。

2. 实现一个”学生成绩”管理系统

答案与深度解析 答案：见代码实现【核心认知框架】：考查集合应用、排序算法、业务逻辑。【设计思路深度解析】

- 数据结构：`ArrayList<Student>` 动态管理学生。
- 排序：`Collections.sort` 配合 Comparator 实现多条件排序。
- 封装：Student 类私有属性，Getter/Setter 控制访问。
- 健壮性：检查 null 和边界条件，避免异常。

【参考代码】

```java
class Student {
    private String id, name;
    private double score;
    // ???getter/setter/toString ??
    public Student(String id, String name, double score) {
        this.id = id; this.name = name; this.score = score;
    }
    public String getId() { return id; }
    public double getScore() { return score; }
    public String toString() { return id + ":" + name + ":" + score;
        }
}

class GradeManager {
    private ArrayList<Student> list = new ArrayList<>();
    public void addStudent(Student s) { list.add(s); }
    public void removeStudent(String id) {
        list.removeIf(s -> s.getId().equals(id));
    }
    public double getAverageScore() {
        return list.stream().mapToDouble(Student::getScore).average().
            orElse(0);
    }
    public List<Student> getTopStudents(int n) {
        list.sort((a, b) -> Double.compare(b.getScore(), a.getScore())
            );
        return list.subList(0, Math.min(n, list.size()));
    }
}
```

【考点延伸】

- **Stream API**：Java 8 流式处理计算平均分。
- **Comparator**：lambda 表达式简化排序逻辑。
- 扩展：可添加文件持久化存储功能。

——试卷结束，祝考试顺利——